

SQA Terms And Definitions

Shared quality-assurance vocabulary for interns and admins.

Audience: SQA testers, admins

Generated: 2026-06-28

SQA Terms And Definitions

Purpose: glossary of Software Quality Assurance terms for interns testing the Quran Academy app.

Core SQA Terms

- Software Quality Assurance (SQA): planned activities that help ensure software meets requirements, works reliably, and is safe to release.
- Quality: how well the system satisfies user needs, business rules, security expectations, performance needs, and maintainability goals.
- Quality control (QC): checking the actual product through testing, reviews, and inspections to find defects.
- Testing: running or inspecting software to discover whether it behaves as expected.
- Verification: checking that the team built the system correctly according to specs, designs, and acceptance criteria.
- Validation: checking that the team built the right system for real users and real workflows.
- Requirement: a documented need or rule the system must satisfy.
- Acceptance criteria: specific conditions that must be true before a feature is considered complete.
- Test basis: the source used to design tests, such as requirements, tickets, designs, user stories, or code behavior.
- Test objective: the reason for a test, such as confirming login routing, checking progress saving, or proving unauthorized access is blocked.

Test Planning

- Test plan: a document describing what will be tested, how it will be tested, who will test it, and what risks matter.
- Test strategy: the overall testing approach for a project or release.
- Test scope: the features, roles, devices, environments, or flows included in testing.
- Out of scope: items intentionally not tested in a given cycle.
- Test environment: the server, database, browser, app build, and configuration used for testing.
- Test data: accounts, students, parents, teachers, sessions, messages, recordings, and other records used during testing.
- Entry criteria: conditions that must be met before testing starts.
- Exit criteria: conditions that must be met before testing can finish.
- Risk-based testing: prioritizing tests based on business impact and likelihood of failure.
- Test coverage: the amount of requirements, code, roles, flows, or risks covered by tests.

Test Design

- Test case: a set of steps, data, expected results, and pass/fail outcome for one behavior.
- Test scenario: a high-level user situation to test, such as "parent opens child messages."
- Test suite: a group of related test cases.
- Test script: detailed instructions that a tester follows exactly.
- Test checklist: a lighter list of items to verify without every click written out.
- Expected result: what should happen if the system works correctly.

- Actual result: what happened during testing.
- Precondition: what must already be true before a test begins.
- Postcondition: what should be true after the test finishes.
- Positive test: confirms the system works with valid inputs and allowed actions.
- Negative test: confirms the system blocks invalid inputs, unauthorized actions, or unsafe states.
- Boundary test: checks values at limits, such as maximum message length or join-window timing.
- Equivalence partitioning: selecting representative inputs from groups that should behave the same.
- Decision table: a table showing combinations of conditions and expected outcomes.
- Traceability: linking tests back to requirements, user stories, tickets, or risks.

Test Execution

- Test run: one execution of a test case or suite.
- Pass: the actual result matches the expected result.
- Fail: the actual result does not match the expected result.
- Blocked: the test cannot be completed because something else prevents it, such as missing account access.
- Retest: running a failed test again after a fix.
- Regression testing: checking that recent changes did not break existing features.
- Smoke testing: quick checks that the build is stable enough for deeper testing.
- Sanity testing: focused checks that a specific fix or small change works.
- Exploratory testing: learning and testing at the same time, following clues and risks instead of only scripted steps.
- Ad hoc testing: informal testing without a prepared plan or script.
- End-to-end testing: testing a complete workflow across multiple components.
- Cross-browser testing: checking behavior in different browsers.
- Compatibility testing: checking behavior across devices, browsers, operating systems, or screen sizes.

Defects And Bug Reports

- Defect: a flaw in the software, requirement, design, data, or configuration.
- Bug: a defect observed in the running system.
- Incident: an observed problem during testing that may or may not become a confirmed bug.
- Bug report: a written record of a defect with steps, evidence, expected result, actual result, and environment.
- Reproduction steps: exact steps needed to make a bug happen again.
- Severity: how much harm the defect causes.
- Priority: how urgently the defect should be fixed.
- Critical defect: a defect that blocks major use, causes data loss, exposes private data, or prevents release.
- Major defect: a serious issue with an important workflow, but with possible workaround.
- Minor defect: a small issue with limited impact.
- Cosmetic defect: a visual or wording issue that does not affect functionality.
- Workaround: an alternate path users can take to complete the task despite a defect.

- Root cause: the underlying reason a defect happened.
- Duplicate bug: a report that describes the same issue as an existing bug.
- Cannot reproduce: a bug report that testers or developers cannot make happen again with the provided steps.

Test Levels

- Unit testing: testing a small function, module, or component in isolation.
- Integration testing: testing how components work together.
- System testing: testing the complete system as a whole.
- User acceptance testing (UAT): confirming the system is acceptable for real users or business owners.
- Production smoke test: quick validation after deployment to production.
- Build verification test (BVT): checks that a new build is usable for further testing.

Test Types

- Functional testing: checks what the system does.
- Non-functional testing: checks qualities such as performance, usability, accessibility, security, reliability, and compatibility.
- UI testing: checks screens, controls, layout, navigation, labels, and visual behavior.
- API testing: checks backend service inputs, outputs, permissions, and error handling.
- Database testing: checks stored data, relationships, updates, and migrations.
- Security testing: checks authentication, authorization, data exposure, and abuse risks.
- Permission testing: checks whether each role can access only what it should.
- Usability testing: checks whether users can complete tasks easily and clearly.
- Accessibility testing: checks whether users with different needs can use the system.
- Performance testing: checks speed, responsiveness, and stability under expected load.
- Load testing: checks system behavior under many users or requests.
- Stress testing: checks system behavior beyond normal expected load.
- Reliability testing: checks whether the system behaves consistently over time.
- Localization testing: checks language, direction, labels, dates, and culturally appropriate content.
- Data migration testing: checks that old data moves correctly into a new schema or system.

Automation Terms

- Test automation: using code or tools to run tests automatically.
- Manual testing: testing performed by a person.
- Automated test: a scripted test that can be run repeatedly by a tool.
- Flaky test: an automated test that sometimes passes and sometimes fails without a real product change.
- Test fixture: setup data or state required by a test.
- Mock: a fake object or service used to simulate a real dependency.
- Stub: a simple replacement that returns fixed responses.

- Test harness: tools and code that run tests and collect results.
- Continuous integration (CI): automatically building and testing code when changes are made.
- Continuous deployment (CD): automatically deploying code after checks pass.

Release And Environment Terms

- Build: a packaged version of the software ready for testing or deployment.
- Deployment: moving software to an environment such as integration, staging, or production.
- Integration environment: early shared environment for testing new changes together.
- Staging environment: production-like environment used before release.
- Production environment: live system used by real users.
- Rollback: returning to a previous working version after a release problem.
- Release candidate: a build that may be released if testing passes.
- Version: a named or numbered software state.
- Configuration: environment settings that affect behavior.
- Feature flag: a setting that turns a feature on or off without changing code.

Evidence And Reporting

- Test evidence: screenshots, videos, logs, exported data, or notes proving what happened.
- Screenshot: image of the screen during testing.
- Console log: browser developer-tool messages, warnings, and errors.
- Server log: backend log messages from the server.
- Network trace: record of browser/API requests and responses.
- Test summary report: final report of what was tested, what passed, what failed, and remaining risks.
- Defect trend: pattern in bug counts, types, severity, or affected areas over time.
- Metrics: numbers used to understand quality, such as pass rate, defect count, or coverage.

Quran Academy Testing Examples

- Role routing: verify student, parent, teacher, and admin accounts land on the correct dashboard.
- Permission testing: verify a parent cannot view another parent's child, and a student cannot view parent-teacher threads.
- Managed progress testing: complete a lesson step, refresh, and confirm the step remains complete.
- Communications testing: verify announcements are read-only and parent-teacher threads allow only authorized participants.
- Live-session testing: verify only permitted users can join a BigBlueButton room during the allowed time window.
- Recording testing: verify Speak and Submit recordings save successfully and appear in the correct report/review area.
- Quiz testing: complete a quiz attempt and confirm attempts, passes, and question results appear in reports.
- Workspace testing: verify materials assigned in a workspace appear only for the correct student or parent.
- Regression testing: after a chat or live-session change, recheck dashboard links, communications, reports, and role permissions.

- Smoke testing: after deployment, confirm login, dashboard, course launch, one unit page, communications, and one report page load without errors.